

```

require("dotenv").config();
const express = require("express");
const cors = require("cors");
const cron = require("node-cron");
const fs = require("fs");
const path = require("path");
const Anthropic = require("@anthropic-ai/sdk");

const anthropic = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });
const MODEL = "claude-sonnet-4-6";

// -----
// Storage – simple JSON file on disk. Good enough for one company's data.
// On some free hosts the disk resets on redeploy; swap this for a real DB
// (e.g. a free Supabase/Postgres instance) once this proves itself out.
// -----
const DATA_DIR = path.join(__dirname, "data");
const DATA_FILE = path.join(DATA_DIR, "store.json");
if (!fs.existsSync(DATA_DIR)) fs.mkdirSync(DATA_DIR, { recursive: true });

function loadStore() {
  try {
    return JSON.parse(fs.readFileSync(DATA_FILE, "utf8"));
  } catch {
    return { gmMessages: [], gmDisplayLog: [], deptLogs: {}, dailyBriefing: null,
  }
}

function saveStore(store) {
  fs.writeFileSync(DATA_FILE, JSON.stringify(store, null, 2), "utf8");
}

let store = loadStore();

function todayKey() {
  const d = new Date();
  return `${d.getFullYear()}-${d.getMonth() + 1}-${d.getDate()}`;
}

// -----
// Company + department configuration (same personas as the artifact version)
// -----
const LOOP_CONTEXT = `Loop Travel & Tourism (انسـتـغـرام Golden Medal يُعرف أيضًا بـ)`;

const COMM_CONTEXT = `Ma بالأتمتة عبر) آلية التواصل الفعلية مع العملاء عبر انسـتـغـرام`

```

```

const DEPTS = {
  finance: {
    name: "المالية", role: "رئيس القسم المالي",
    sheetUrl: "https://docs.google.com/spreadsheets/d/1lOFhWpKIUd3kV7brNa-6BCJQmb",
    sheetColumns: "سعر 3 أشخاص، سعر 4 أشخاص، ملاحظات",
    system: `Loop Travel & Tourism. ${LOOP_CONTEXT}
    أنت رئيس القسم المالي في`
    صيغة "نحن" لا "أنتم"، وتقدم تقاريرك مباشرة للمدير العام، Loop أنت عضو فعلي من فريق
    جهات السياحة، الميزانيات، التدفق النقدي، وتقييم الجدوى المالية للمبادرات الجديدة
    الفعلية بدل اختلافاً. أجب بالعربية بإيجاز تنفيذي (لا تتجاوز فقرة أو فقرتين قصيرتين)
  },
  marketing: {
    name: "التسويق", role: "رئيس قسم التسويق",
    sheetUrl: null, sheetColumns: null,
    system: `Loop Travel & Tourism. ${LOOP_CONTEXT}
    أنت رئيس قسم التسويق في`
    صيغة "نحن" لا "أنتم"، وتقدم تقاريرك مباشرة للمدير العام، Loop أنت عضو فعلي من فريق
    محتوى، الإعلانات الممولة، هوية العلامة، وتحسين معدلات التحويل من المتابعين إلى حجوزات
    سويق فعلي، مقترحات عملية وقابلة للتنفيذ، بالعربية، بإيجاز تنفيذي (فقرة أو فقرتين)
  },
  strategy: {
    name: "الاستراتيجية", role: "رئيس قسم الاستراتيجية",
    sheetUrl: null, sheetColumns: null,
    system: `Loop Travel & Tourism. ${LOOP_CONTEXT}
    أنت رئيس قسم الاستراتيجية في`
    صيغة "نحن" لا "أنتم"، وتقدم تقاريرك مباشرة للمدير العام، Loop أنت عضو فعلي من فريق
    وسع، الشراكات، تحليل المنافسين والسوق، وأولويات المشروع على المدى المتوسط والبعيد
    س استراتيجية فعلي، بخطوات واضحة الأولوية، بالعربية، بإيجاز تنفيذي (فقرة أو فقرتين)
  },
  communication: {
    name: "الاتصال", role: "رئيس قسم الاتصال",
    sheetUrl: null, sheetColumns: null,
    system: `Loop Travel & Tourism. ${LOOP_CO
    أنت رئيس قسم الاتصال وخدمة العملاء في`
    صيغة "نحن" لا "أنتم"، وتقدم تقاريرك مباشرة للمدير العام، Loop أنت عضو فعلي من فريق
    ${COMM_CONTEXT}
    وتحسين تجربة العميل في المحادثات - بما ينسجم مع آلية الأتمتة الفعلية الموضحة أعلاه
    ،`. أجب كرئيس اتصال فعلي، بالعربية، بإيجاز تنفيذي (فقرة أو فقرتين)
  },
};

```

استخدام أي أداة Loop Travel & Tourism أنت المدير العام لشركة`
 \${LOOP_CONTEXT}
 ، وادمجها في توصية واحدة متماسكة - وأنت من يملك القرار النهائي بصفتك المدير العام
 رقائماً أو حقائق عن الشركة؛ إن لم تكن المعلومة متوفرة، وضح ذلك. تحدث بالعربية دائماً
 معلومة أو التوصية نفسها - بدون أي مقدمة تشرح خطواتك أو تسرد أنك استشرت قسماً معيناً

بأعمال الشركة Loop. أنت السكرتير الشخصي لنواف داخل منصة`
 const SECRETARY_SYSTEM = (web_search): نفذ الخطوات التالية باستخدام أداة البحث على الإنترنت
 1. ونسبة تغيره خلال 24 ساعة (BTC/USD) سعر البيتكوين الحالي.
 2. وتغيره اليومي (للأونصة XAU/USD) سعر الذهب الحالي.

3. وتغيره اليومي (دولار للبرميل، WTI خام) سعر النفط الحالي.
4. ETH 0.1866579 (تكلفة 2319.68)، SOL 4.49919209 (84.2080)، ATOM 196. محفظة نواف.
5. سبت/أحد → رأس الخيمة، غير ذلك → أبوظبي: (UTC+4) اليوم في الخليج.
6. طقس تلك المدينة اليوم: أعلى/أقل حرارة، الحالة، احتمال الأمطار.
7. أجب مباشرة بالشكل التالي بالضغط، بدون أي مقدمة أو سرد لخطوات البحث:

☀ صباح الخير

*(خلال 24 ساعة [%+/-X.XX]) بيتكوين: \$[السعر]**

*([%+/-X.XX]) الذهب: \$[السعر]/أونصة**

*([%+/-X.XX]) برميل/[السعر] \$ (WTI): النفط**

*(%+/-X.XX) محفظتك**

، أقل درجة حرارة حوالي °[X] الطقس - [المدينة]: ** [الحالة]، أعلى درجة حرارة حوالي**

```
const TOOLS = Object.entries(DEPTS).map(([id, d]) => ({
  name: `consult_${id}`,
  description: `بمهمة أو سؤال يقع ضمن مسؤولياته Loop Travel في تكليف ${d.role}`,
  input_schema: {
    type: "object",
    properties: { instruction: { type: "string", description: "جّه لرئيس هذا القسم",
      required: ["instruction"],
    },
  },
}));
```

```
// -----
// Claude call helpers
// -----
function textOf(content) {
  return (content || []).filter((b) => b.type === "text").map((b) => b.text).join
}
function finalTextOf(content) {
  const full = textOf(content);
  const marker = "صباح الخير";
  const idx = full.indexOf(marker);
  return idx >= 0 ? full.slice(idx).trim() : full;
}
```

```
async function callClaude({ system, messages, tools }) {
  const res = await anthropic.messages.create({
    model: MODEL, max_tokens: 1000, system, messages, ...(tools ? { tools } : {})
  });
  return res;
}
```

```

async function callDepartment(deptId, instruction) {
  const dept = DEPTS[deptId];
  let systemText = dept.system;
  if (dept.sheetUrl) {
    try {
      const res = await fetch(dept.sheetUrl);
      if (res.ok) {
        const csv = (await res.text()).split("\n").slice(0, 80).join("\n");
        systemText += ` \n\nملف من Loop بيانات فعلية حديثة من ملف (تنسيق CSV، الأعمدة: ${d
      } else {
        systemText += ` \n\n(التخمين) إن احتاجه السؤال بدل التخمين)
      }
    } catch {
      systemText += ` \n\n(التخمين) إن احتاجه السؤال بدل التخمين)
    }
  }
  try {
    const res = await callClaude({ system: systemText, messages: [{ role: "user",
    return textOf(res.content) || "لا يوجد رد.";
  } catch {
    return "تعذر الوصول للقسم حاليًا.";
  }
}

function appendDeptLog(deptId, instruction, response) {
  if (!store.deptLogs[deptId]) store.deptLogs[deptId] = [];
  store.deptLogs[deptId].push({ instruction, response, ts: Date.now() });
  saveStore(store);
}

async function runGM(userText) {
  let messages = [...store.gmMessages.slice(-16), { role: "user", content: userTe
  let finalText = null;
  let consultedAll = [];

  for (let round = 0; round < 3 && finalText === null; round++) {
    const res = await callClaude({ system: GM_SYSTEM, messages, tools: TOOLS });
    const content = res.content || [];
    const toolUses = content.filter((b) => b.type === "tool_use");
    messages = [...messages, { role: "assistant", content }];

    if (toolUses.length === 0) { finalText = textOf(content) || "نم."; break; }

    const toolResults = [];
    for (const tu of toolUses) {
      const deptId = tu.name.replace("consult_", "");

```

```

        const instruction = tu.input?.instruction || "";
        const responseText = await callDepartment(deptId, instruction);
        appendDeptLog(deptId, instruction, responseText);
        consultedAll.push(deptId);
        toolResults.push({ type: "tool_result", tool_use_id: tu.id, content: respon
    }
    messages = [...messages, { role: "user", content: toolResults }];
}

store.gmMessages = messages.slice(-16);
const reply = finalText || "تم تنفيذ طلبك.";
const depts = [...new Set(consultedAll)];
store.gmDisplayLog.push({ role: "user", text: userText, ts: Date.now() });
store.gmDisplayLog.push({ role: "gm", text: reply, depts, ts: Date.now() });
store.gmDisplayLog = store.gmDisplayLog.slice(-100);
saveStore(store);
return { reply, depts };
}

async function generateDailyBriefing() {
    const result = await runGM("أقسام الأربعة حسب الحاجة، وأبرز ما يستحق انتباهي الآن");
    store.dailyBriefing = { text: result.reply, depts: result.depts, ts: Date.now() };
    saveStore(store);
    return store.dailyBriefing;
}

async function generateSecretaryBriefing() {
    const res = await callClaude({
        system: SECRETARY_SYSTEM,
        messages: [{ role: "user", content: "أعطني الإحاطة الصباحية" }],
        tools: [{ type: "web_search_20250305", name: "web_search" }],
    });
    const text = finalTextOf(res.content) || "تعذر تجهيز الإحاطة الشخصية اليوم.";
    store.secretaryBriefing = { text, ts: Date.now(), day: todayKey() };
    saveStore(store);
    return store.secretaryBriefing;
}

// -----
// HTTP API
// -----

const app = express();
app.use(cors({ origin: process.env.ALLOWED_ORIGIN || "*" }));
app.use(express.json());

app.get("/api/health", (req, res) => res.json({ ok: true, time: new Date().toISOString

```

```

app.get("/api/state", (req, res) => {
  res.json({
    gmDisplayLog: store.gmDisplayLog,
    deptLogs: store.deptLogs,
    dailyBriefing: store.dailyBriefing,
    secretaryBriefing: store.secretaryBriefing,
  });
});

app.post("/api/chat", async (req, res) => {
  const message = (req.body?.message || "").trim();
  if (!message) return res.status(400).json({ error: "message is required" });
  try {
    const result = await runGM(message);
    res.json(result);
  } catch (e) {
    res.status(500).json({ error: e.message || "internal error" });
  }
});

app.post("/api/department/:id", async (req, res) => {
  const { id } = req.params;
  const message = (req.body?.message || "").trim();
  if (!DEPTS[id]) return res.status(404).json({ error: "unknown department" });
  if (!message) return res.status(400).json({ error: "message is required" });
  try {
    const responseText = await callDepartment(id, message);
    appendDeptLog(id, message, responseText);
    res.json({ reply: responseText });
  } catch (e) {
    res.status(500).json({ error: e.message || "internal error" });
  }
});

app.get("/api/briefing/daily", async (req, res) => {
  try {
    if (store.dailyBriefing?.day === todayKey()) return res.json(store.dailyBrief);
    const fresh = await generateDailyBriefing();
    res.json(fresh);
  } catch (e) {
    res.status(500).json({ error: e.message || "internal error" });
  }
});

app.post("/api/briefing/daily/refresh", async (req, res) => {
  try {
    res.json(await generateDailyBriefing());
  }
});

```

```

    } catch (e) {
      res.status(500).json({ error: e.message || "internal error" });
    }
  });

app.get("/api/briefing/secretary", async (req, res) => {
  try {
    if (store.secretaryBriefing?.day === todayKey()) return res.json(store.secret
    const fresh = await generateSecretaryBriefing();
    res.json(fresh);
  } catch (e) {
    res.status(500).json({ error: e.message || "internal error" });
  }
});

app.post("/api/briefing/secretary/refresh", async (req, res) => {
  try {
    res.json(await generateSecretaryBriefing());
  } catch (e) {
    res.status(500).json({ error: e.message || "internal error" });
  }
});

app.post("/api/reset", (req, res) => {
  store = { gmMessages: [], gmDisplayLog: [], deptLogs: {}, dailyBriefing: null,
  saveStore(store);
  res.json({ ok: true });
});

// -----
// Scheduled jobs – this is the actual "runs while you're asleep" part.
// 3:00 UTC = 7:00 AM Gulf Standard Time (UTC+4).
// -----
cron.schedule("0 3 * * *", async () => {
  console.log("[cron] generating daily briefing...");
  try { await generateDailyBriefing(); } catch (e) { console.error("[cron] daily
  console.log("[cron] generating secretary briefing...");
  try { await generateSecretaryBriefing(); } catch (e) { console.error("[cron] se
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Loop GM server running on port ${PORT}`));

```